

Real-Time Cybersecurity Event Analytics Platform

Group 4- B23EE1094, B23ES1032, B23EE1041

1. Technical Analysis of Existing Market Solutions

Our problem domain focuses on real-time security event analytics within the Security Information and Event Management (SIEM) space. The two most relevant commercial solutions are Splunk Enterprise Security and Elastic Security, both dominant enterprise platforms that ingest log data, detect threats and render security dashboards

A. Splunk Enterprise Security: Splunk operates on a “schema-on-read” model where incoming logs are stored largely as raw text and field structures are extracted dynamically when a search query runs. Scheduled correlation searches, which drive most of Enterprise Security's detection workflows, are index/search based. Splunk also supports real-time searches that can evaluate events before indexing, but this mode is resource-intensive at scale and is not equivalent to a purpose-built continuous stream-processing detection engine. A related indexed real-time search mode introduces a default ~60-second synchronization delay.

❖ Pros:

- very flexible when dealing with non-standard or unformatted log sources
- powerful querying through Search Processing Language (SPL)
- huge ecosystem of pre-built integrations.

❖ Cons:

- heavy compute cost since parsing happens at query time
- ingestion pipeline is less transparent and less directly controllable than a custom-built streaming pipeline
- detection responsiveness in scheduled/index-based workflows is bounded by the search or indexing cycle rather than event arrival

B. Elastic Security: Built on the ELK stack (Elasticsearch, Logstash, Kibana). Logstash continuously ingests and normalizes log streams into a common schema, after which Elastic's Detection Engine evaluates rules on configurable schedules over stored Elasticsearch indexes, rather than evaluating every incoming event through a dedicated continuous stream-processing engine.

❖ Pros:

- open and relatively transparent architectural components
- flexible detection rule types

- strong long-term log retention through hot, warm, and frozen index tiers.
- ❖ Cons:
 - detections remain fundamentally index-and-query based, making them periodic rather than instantaneous
 - running rules over large datasets gets resource-heavy
 - operating it at scale is complex.

Both existing solutions follow a “store-first, query-later” paradigm by default, where events are generally written to an index before scheduled or periodic searches analyze them. Splunk's scheduled correlation/detection workflows and Elastic's rule-based detection engine are both primarily index/search oriented. While Splunk's real-time search can evaluate events before indexing, it is not a dedicated, purpose-built continuous stream-processing engine, a key architectural gap our project addresses directly.

2. Architectural & Functional Comparison with Our Solution

Aspect	Splunk / Elastic (industry)	Our Platform
Data source	Passive and waits for real production logs	Active and built-in simulator generates controllable normal/attack scenarios
Detection model	Store first, then query (scheduled/indexed)	Stream first and detects on events as they flow through
Pipeline visibility	Mostly closed/proprietary internals	Fully transparent, custom-built ETL/ELT
Storage	Single index (tiered internally)	Separate data lake (raw) + data warehouse (aggregated)
Monitoring	Only security alerts/dashboards	Security alerts and pipeline health (latency, throughput)

3. Gaps our project addresses

- **Integrated Closed-Loop Simulation Controller:** Commercial SIEMs generally assume production environments already supply steady log streams, requiring external tools to test detection logic. Our proposed architecture embeds a controllable event generator

alongside the ingestion pipeline, allowing users to trigger specific attack vectors (e.g., brute-force login spikes and abnormal traffic) and immediately observe transformation and detection in real time.

- **Stream-First Anomaly Detection by Design:** Rather than relying primarily on writing events to disk and waiting for periodic search jobs, our proposed streaming engine is designed to evaluate detection rules in-flight as data arrives, targeting lower detection latency by architectural design.
- **Transparent, Decoupled Storage Architecture:** By explicitly separating raw log retention (data lake) from aggregated metric storage (data warehouse), our design aims to reduce heavy index-lookup overhead, keep dashboard queries responsive, and maintain full forensic event history
- **Pipeline Health & Reliability Diagnostics:** Most security platforms focus primarily on security monitoring. Our proposed architecture also treats stream processing as a core engineering concern, with planned tracking of event latency, throughput, queue bottlenecks, and backpressure to assess system stability during high-volume attack bursts.

4. References

1. Kreps, J. (2014). *Questioning the Lambda Architecture*. O'Reilly Radar.
2. Carbone, P., et al. (2015). *Apache Flink: Stream and Batch Processing in a Single Engine*. IEEE Data Engineering Bulletin, 38(4), 28–38.
3. González-Granadillo, G., et al. (2021). *Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures*. Sensors, 21(14), 4759. <https://doi.org/10.3390/s21144759>
4. Chandola, V., Banerjee, A., Kumar, V. (2009). *Anomaly Detection: A Survey*. ACM Computing Surveys, 41(3), 1–58. <https://doi.org/10.1145/1541880.1541882>
5. Google Patent US20110016123A1. *Scalable Real Time Event Stream Processing*.
6. Splunk Docs. *Expected performance and known limitations of real-time searches and reports*.
7. Elastic Docs. *Detections and alerts*.

Opportunity

Solution Needed

- ✓ Solution identified
- ✓ Stakeholders' needs established
- ✓ Problems and root causes identified
- ✓ Need for a solution confirmed
- ✓ At least one solution proposed

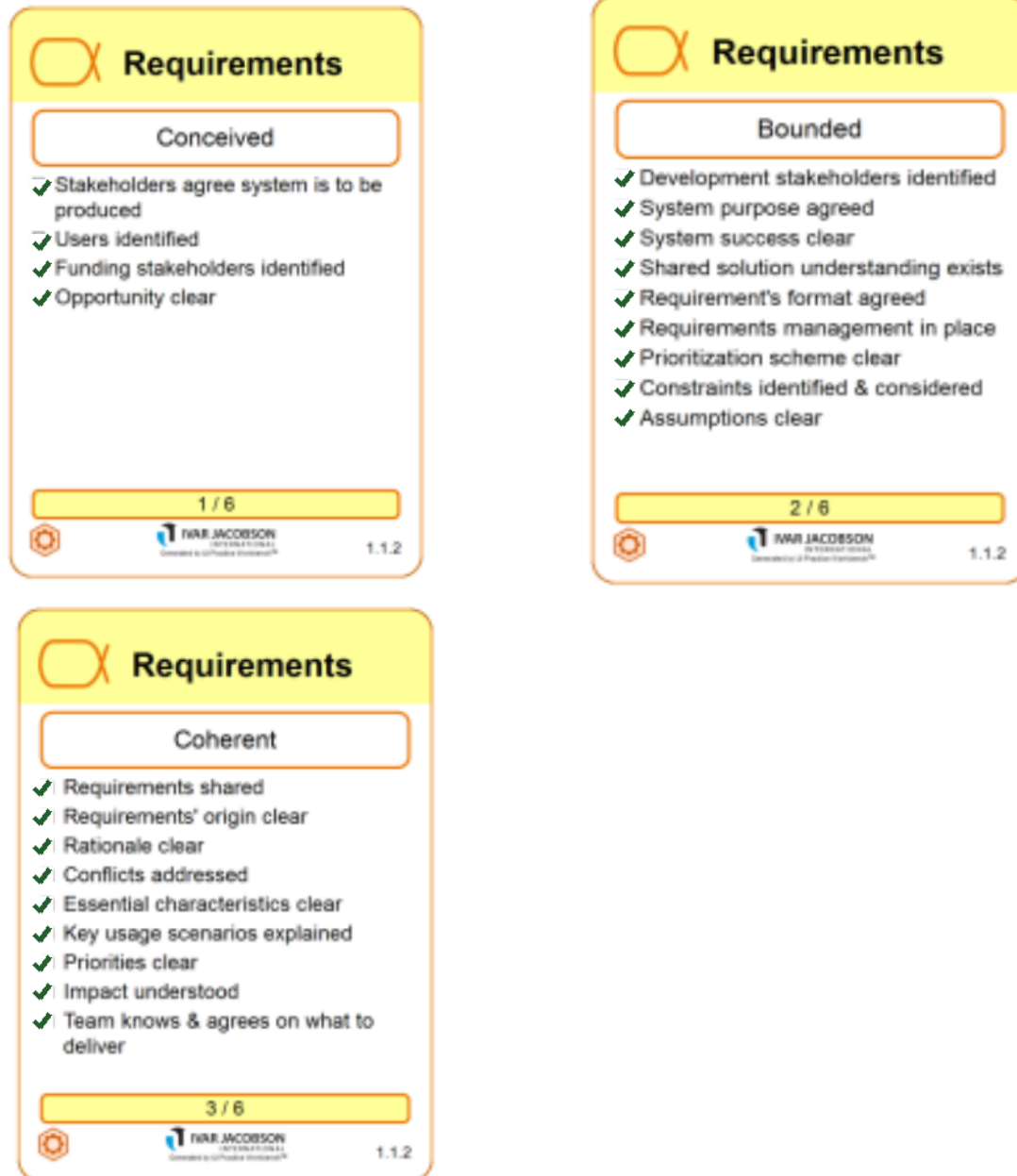
2 / 6

  **NIAR JACOBSON**
DEFENDING YOUR BUSINESS
Sustained by 500+ Positive Endorsements™

1.1.2

Research was carried out to understand the existing market solutions and their technical approaches. From this study, the market gap and the value our project could add were identified, leading to the proposed solution.

Requirements



Based on the analysis and discussion, the functional requirements were decided and divided among the team members so that each requirement has clear ownership and shared responsibility where needed.

Functional Requirements

S.No.	What the system is expected to do	Priority	Risk Analysis
FR1	Generate normal and attack logs like brute force, api hits and port scan from simulated sources	High	Generator might lag or drop events under heavy load
FR2	Live stream processing to catch attack patterns with sliding windows and trigger quick alerts	High	Out of order events causing high memory usage and missed alerts
FR3	Dual data storage saving raw json logs to data lake and clean aggregated data to warehouse	High	Schema changes in incoming logs breaking warehouse inserts
FR4	Interactive ui controls to inject custom attack scenarios on demand	Medium	Sudden flood of attack traffic could crash the simulator frontend
FR5	Real time monitoring dashboard to show log graphs, detected anomalies and pipeline stats	Medium	Frequent auto polling slowing down backend analytical queries

Responsibility Split

Team Members	Responsibility_1	Responsibility_2
Member_1 (Maulik)	FR1	FR5
Member_2 (Sudhanshu)	FR2 (with Satvik)	FR4
Member_3 (Satvik)	FR2 (with Sudhanshu)	FR3